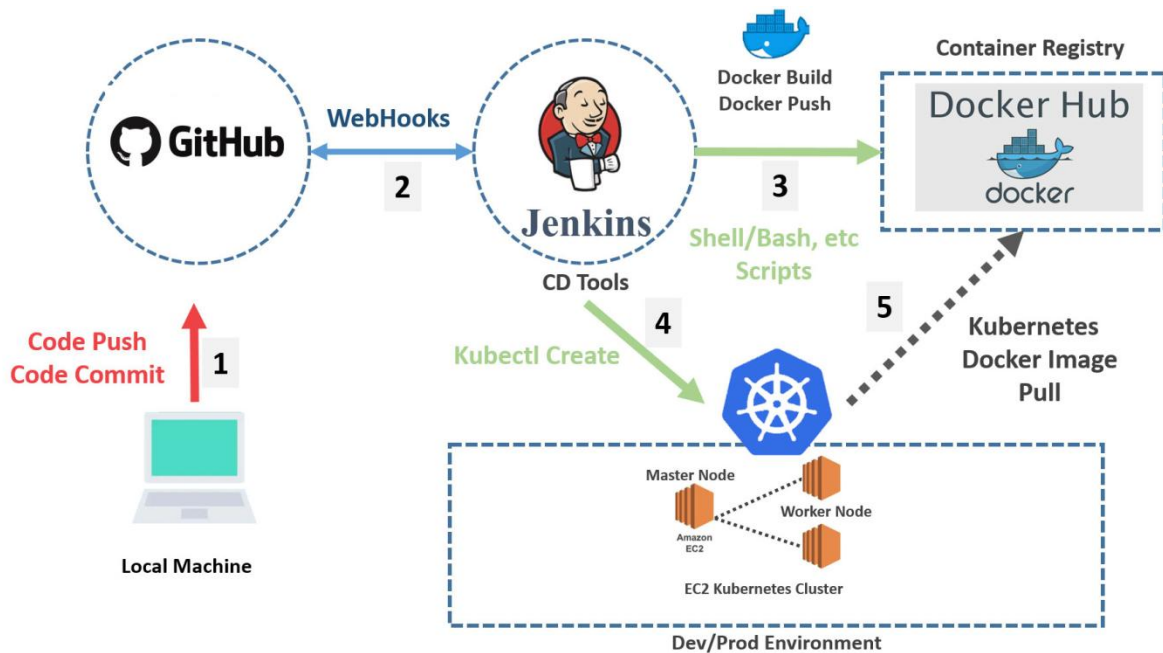


Writing code to deploying a running application



Here's a step-by-step breakdown of the CI/CD pipeline shown in the diagram:

1. Code Push / Code Commit:

- Developers write and push their code to a **GitHub repository**.

2. Webhook Notification:

- GitHub** sends a webhook notification to **Jenkins**, informing it of the new code commit.

3. Docker Image Build & Push:

- Jenkins triggers a **Continuous Deployment (CD) pipeline** that:
 - Builds a **Docker image** from the latest source code.
 - Pushes the image to **Docker Hub**, a container registry.

4. Deploying to Kubernetes via `kubectl create`:

- Jenkins uses **Kubernetes command-line tool (kubectl)** to create or update resources in the **Kubernetes cluster**.

5. Kubernetes Image Pull & Deployment:

- Kubernetes** pulls the **Docker image** from **Docker Hub** and deploys the containerized application on an **EC2 Kubernetes cluster** in a **development/production environment**.

This pipeline automates the entire process—from writing code to deploying a running application—making software delivery **efficient, scalable, and reliable**.

Deploying a containerized application in a local Kubernetes cluster with Minikube

1. Start Minikube: `minikube start`

- Minikube is a lightweight Kubernetes implementation that runs on your local machine.
- This command initializes a single-node Kubernetes cluster.
- It sets up the required VM or container runtime to host Kubernetes.

2. Check Minikube Status: `minikube status`

- This command checks if Minikube is running and provides details like:
 - The status of Minikube's VM or container.
 - The status of the Kubernetes API server.
 - The status of configured Kubernetes services.

3. Run a Pod with Your Docker Image

`kubectl run demodockuber --image=naresh5021/dockuberdemo:1.0 --port=80`

- This command creates a **Pod** named demodockuber.
- It pulls the image naresh5021/dockuberdemo:1.0 from Docker Hub (or any specified registry).
- The container inside the Pod is set to expose port 80.

4. Check Running Pods: `kubectl get pods`

- This retrieves a list of all running Pods in the cluster.
- You can check their status (Running, Pending, Failed, etc.).
- Useful for confirming that demodockuber is successfully created and running.

5. Expose the Pod as a Service

`kubectl expose pod demodockuber --name=demodockubersvc --port=80`

- This command creates a **Kubernetes Service** named demodockubersvc.
- It enables network access to the demodockuber Pod on port 80.
- By default, it uses ClusterIP, meaning the service is accessible only within the cluster.

6. Check Available Services: `kubectl get svc`

- This lists all running services in the Kubernetes cluster.
- It includes details like service name, cluster IP, exposed ports, and type (ClusterIP, NodePort, LoadBalancer).

7. Forward Traffic from Local to Kubernetes Service

```
kubectl port-forward service/demodockubersvc 8181:80
```

- This maps local port 8181 to the Kubernetes service demodockubersvc (which exposes port 80).
- You can now access your application in the browser or via curl using: **http://localhost:8181**
- This is useful when running Minikube, as ClusterIP services aren't directly accessible externally.

By completing these steps, you're successfully deploying a containerized application in a local Kubernetes cluster with Minikube!

```
kubectl create deployment jnkapp2 --image=naresh5021/csu2:v1
kubectl get deployment
kubectl scale deployment jnkapp2 --replicas=10
kubectl get svc
kubectl get pods
kubectl expose deployment jnkapp2 --name=demodockuber2svc --port=80 --
type=ClusterIP
kubectl describe deployment jnkapp2
kubectl edit deployment jnkapp2
```

Docker Compose :

Docker Compose is a tool for defining and running multi-container applications with a single YAML file.

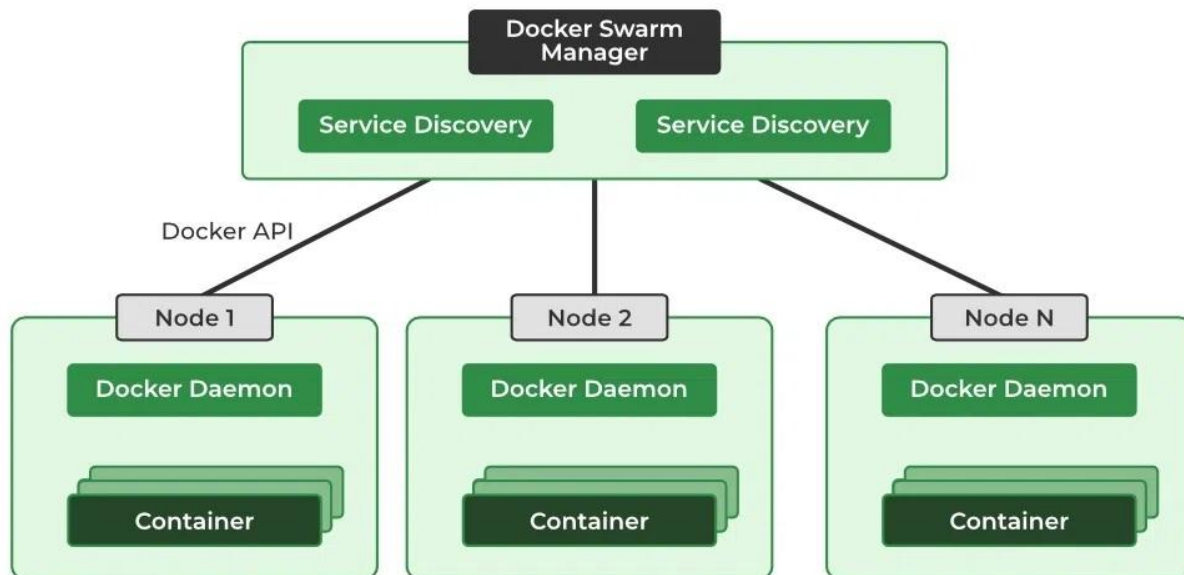
Compose simplifies the control of your entire application stack, making it easy to manage services, networks, and volumes in a single YAML configuration file. Then, with a single command, you create and start all the services from your configuration file.

Docker Swarm :

Docker Swarm is the container orchestration tool that manages all the containers available in the Swarm cluster.

Swarm Mode in Docker was introduced in version 1.12, enabling the ability to deploy multiple containers on multiple Docker hosts. For this Docker uses an overlay network for the service discovery and with a built-in load balancer for scaling the services.

One of the key advantages of docker swarm is that the configuration of the docker services such as volume and network can be modified without the need to manually restart the service Docker will update the configuration, stop the service tasks with the out-of-date configuration and create new ones matching the desired configuration.



Minikube & Kubernetes :

Kubernetes is designed for large-scale deployments across multiple nodes and clusters, making it suitable for managing complex and distributed environments. On the other hand, Minikube is a lightweight and simplified version of Kubernetes, primarily used for local development and testing purposes. It allows developers to run a single-node Kubernetes cluster on their local machines. It supports various virtualization technologies such as VirtualBox, VMware, and Docker.

Kubectl:

Kubectl is a command-line tool used to interact with Kubernetes clusters. It allows you to deploy applications, inspect and manage cluster resources, and view logs. Kubectl is an essential tool for managing Kubernetes clusters, whether they are running locally (e.g., with Minikube) or in a production environment.